

Cloud Computing TASK – 3

- 1. Set up a free-tier AWS account (or use an AWS Cloud sandbox) and launch a basic EC2 instance that serves a static HTML page. Document the steps you took and the public URL of your instance.**

Ans :

- Create a free AWS account (or log in to your AWS Cloud Sandbox).
 - Open the AWS Management Console.
 - Navigate to EC2 from the Services menu.
 - Choose an AWS Region (e.g., US East (N. Virginia)).
 - Click Launch Instance.
 - Enter an instance name (e.g., Static-Web-Server).
 - Select Amazon Linux 2023 AMI (Free Tier Eligible).
 - Choose the instance type t2.micro or t3.micro.
 - Create a new key pair or select an existing one and download the .pem file.
 - Configure the security group:
 - Allow SSH (Port 22) from My IP.
 - Allow HTTP (Port 80) from Anywhere (0.0.0.0/0).
 - Leave the storage settings as default (8 GB).
 - Click Launch Instance.
- 2. Create an Auto Scaling Group in AWS that maintains a minimum of 1 and a maximum of 3 EC2 instances for your web server. Set the scaling policy to add a new instance if CPU utilization goes above 60% for 5 minutes.

Hint: Use the AWS Management Console's Auto Scaling Groups section and attach your EC2 launch template or configuration.**

Ans :

- Log in to the AWS Management Console.
 - Navigate to EC2.
 - In the left menu, click Launch Templates.
 - Click Create Launch Template.
 - Enter a template name (e.g., WebServerTemplate).
 - Select the same Amazon Machine Image (AMI) used for your web server (e.g., Amazon Linux 2023).
 - Choose the instance type (t2.micro or t3.micro).
 - Select your existing key pair.
 - Select the security group that allows:
 - SSH (Port 22)
 - HTTP (Port 80)
 - Click Create Launch Template.
 - In the left menu, select Auto Scaling Groups.
 - Click Create Auto Scaling Group.
 - Enter a name (e.g., WebServer-ASG).
 - Select the launch template you created.
 - Click Next.
 - Choose the VPC and one or more subnets.
 - Click Next.
 - Skip attaching a load balancer (or attach one if required).
 - Under Group Size, configure:
 - Desired Capacity: 1
 - Minimum Capacity: 1
 - Maximum Capacity: 3
- 3. Simulate increased traffic to your web server by using an online load testing tool (such as Loader.io or k6) and observe if**

the Auto Scaling Group launches additional instances when CPU usage spikes.

Ans :

- Ensure your EC2 instance and Auto Scaling Group (ASG) are running.
- Confirm the ASG settings:
- Minimum Capacity: 1
- Desired Capacity: 1
- Maximum Capacity: 3
- Scaling Policy: Add 1 instance when CPU utilization > 60% for 5 minutes.
- Copy the Public URL of your web server (or the Load Balancer URL if using one).
- Open an online load testing tool such as Loader.io or install k6.
- Create a new load test targeting your web server URL.
- Configure the test with increasing concurrent users or requests (e.g., 100–500 virtual users for 5–10 minutes).
- Start the load test.
- Open the AWS Management Console and navigate to EC2 → Auto Scaling Groups.
- Monitor the Activity tab of the Auto Scaling Group for scaling events.
- Open CloudWatch → Metrics → EC2 and observe the CPUUtilization metric.
- Wait for CPU utilization to remain above 60% for at least 5 minutes.
 - Verify that the Auto Scaling Group launches an additional EC2 instance.

4. After scaling up, reduce the simulated load and verify that the Auto Scaling Group scales the number of instances back down to the minimum. Take screenshots of the scaling activity history as evidence.

Ans :

- Ensure the Auto Scaling Group has already scaled up (e.g., from 1 to 2 or 3 EC2 instances) during the load test.
- Stop or significantly reduce the traffic generated by your load testing tool (e.g., Loader.io or k6).
- Open the AWS Management Console.
- Navigate to EC2 → Auto Scaling Groups.
- Select your Auto Scaling Group.
- Open the Activity tab to monitor scaling events.
- Navigate to CloudWatch → Metrics → EC2 and observe that CPUUtilization decreases below the scaling threshold.
- Wait for the Auto Scaling Group to complete the scale-in process according to its configured cooldown and scale-in policy.
- Verify that the number of running EC2 instances decreases back to the minimum capacity of 1.

5. Use ChatGPT or another AI tool to generate a list of 3 real-world scenarios (from apps like Zomato, Flipkart, or BookMyShow) where auto-scaling is essential, and explain in your own words why auto-scaling helps in each scenario.

Ans :

1. Zomato – Meal Ordering During Peak Hours

Scenario: During lunch (12 PM–2 PM) and dinner (7 PM–10 PM), thousands of users browse restaurants and place orders at the same time.

Why Auto Scaling Helps:

Auto scaling automatically launches additional servers to handle the sudden increase in user requests. This keeps the app responsive, reduces delays, and ensures customers can place orders without the website or app slowing down. Once demand decreases, the extra servers are removed to reduce costs.

2. Flipkart – Big Billion Days Sale

Scenario: During major sales such as Big Billion Days, millions of customers visit the website simultaneously to browse products and make purchases.

Why Auto Scaling Helps:

Auto scaling increases the number of servers during the sale to manage the heavy traffic and prevent crashes. After the sale or when traffic returns to normal, it scales down the servers, helping the company save infrastructure costs while maintaining good performance.

3. BookMyShow – Popular Movie or Concert Ticket Booking

Scenario: When ticket sales open for a blockbuster movie or a highly anticipated concert, a large number of users try to book tickets at the same time.

Why Auto Scaling Helps:

Auto scaling adds more servers to handle the spike in booking requests, reducing waiting times and preventing the application from becoming unavailable. After ticket demand decreases, it automatically removes the extra servers, ensuring efficient resource usage and lower operating costs.

